Bangladesh University of Business and Technology (BUBT)

PROJECT PROPOSAL

on

Smart City Route & Infrastructure Planner

Submission Date: **November 16, 2026**

Submitted by:

ID	Name	Intake/Section
20254103279	MAKHZUM-BIN-HARUN	56/07
20254103248	OALID KHAN	56/07
20254103245	MUNTAHA MOU JIM	56/07
20254103269	MD. RAFIUL ISLAM	56/07

Submitted to

Mushfiqur Rahman

Lecturer, Dept. of CSE

BUBT

01. NEEDS/PROBLEMS

A city road network connects many locations through many roads. Answering simple questions about such a network by hand quickly becomes difficult:

- **Finding the shortest route is not straightforward**. When many locations are connected by many roads, it is hard to work out by hand which route between two places is the shortest.
- **Connectivity is hard to check**. It is not obvious which locations can be reached from a given place, or whether some parts of the network are cut off from the rest.
- **Choosing which roads are essential is difficult**. Without a systematic method, it is hard to find the smallest total road length that still keeps all locations connected.
- **Network data is difficult to manage**. Location and road information kept on paper or in unorganized files is hard to update, search and reuse.
- **Planning facility locations is difficult**. Deciding where to place a hospital, fire station or emergency center so that every location can be reached quickly requires the shortest distances between all pairs of locations, which is impractical to work out by hand.
- **Graph algorithms are often studied only on abstract examples**. Students rarely see how these algorithms apply to a familiar problem such as a city road network.

This project is an **educational simulation**. It does not aim to be a real-world urban planning tool. Instead, it shows how classic graph algorithms can be applied to a **simplified city road network**. The program represents the network as a **weighted graph** and uses graph algorithms to explore it, find shortest routes between two locations and between all pairs of locations, identify a minimum set of roads that keeps the network connected, and suggest the best location for a facility.

02. GOALS/OBJECTIVES

SmartCity is a console-based application developed in **C++17**. The aim is to build a working, easy-to-understand program that models a simplified city road network and demonstrates core graph algorithms on it. The program runs offline and uses only free tools.

The specific objectives of this project are:

1. Represent the city road network as a **weighted, undirected graph** using an **adjacency list**, where locations are vertices and roads are edges weighted by distance in kilometers.
2. Allow the user to add, view, update and delete **locations** and **roads**, with input validation that keeps the network consistent.
3. Display the road network as an **adjacency list**: each location with its connected roads and distances.
4. Explore connectivity using **Breadth-First Search (BFS)** and **Depth-First Search (DFS)**.
5. Find the shortest route between two locations using **Dijkstra's algorithm**.
6. Find a minimum road network using **Kruskal's algorithm** with a **Disjoint Set Union (DSU)**.
7. Find the shortest distances between **all pairs** of locations using the **Floyd–Warshall algorithm**, and rebuild the route between any two locations.
8. Suggest the **best facility location** (the network center) from each location's **eccentricity**, and report the network **radius** and **diameter**, handling disconnected networks and isolated locations correctly.

9. Save and load the network data using **CSV files**, starting from a sample Dhaka network on the first run.
10. Provide a clear, menu-driven **console interface** with input validation and helpful error messages.
11. Test the main functions using **automated tests** that compare results with known correct answers (400 automated checks in the final version).

03. EXISTING SYSTEM ANALYSIS

Before designing this system, we looked at existing tools and approaches related to route finding, infrastructure planning and graph algorithms, to understand their purpose, their limitations for our use case and how our project differs.

System	Type	Key Limitations Relevant to Our Project
Google Maps	Online navigation service	Designed for real-world navigation on real map data. The user cannot build or edit their own road network, the route-finding method is not shown, and it does not answer planning questions such as the minimum road network or the best facility location for a user-defined network.
VisuAlgo	Educational algorithm visualization website	Shows individual algorithms step by step on small example graphs. Each algorithm is studied separately; there is no saved, user-managed city network and no combined planning workflow (routes, minimum network, facility location) on the same data.
Manual planning (paper or spreadsheets)	Manual records	Distances, connectivity and routes must be worked out by hand. This is slow and error-prone, especially for all pairs of locations, and must be repeated whenever a road or location changes.
Basic shortest-path program	Single-algorithm program (e.g. a Dijkstra exercise)	Answers only one question: the route between two locations. It does not check connectivity, build a minimum road network, compare all pairs of locations or suggest a facility location, and often has no data management or input validation.

These tools serve their own purposes well. SmartCity addresses a different, more specific use case: an **offline, educational program** in which the user builds and maintains their own simplified city road network and applies several algorithmic approaches to the **same data**:

- **Network traversal**: BFS and DFS show which locations can be reached from a chosen location.
- **Shortest-path analysis**: Dijkstra's algorithm finds the shortest route between two locations.
- **Minimum infrastructure connectivity**: Kruskal's algorithm with a Disjoint Set Union selects the roads with the smallest total length that keep the network connected.

• **All-pairs shortest-path analysis**: the Floyd–Warshall algorithm computes the shortest distance between every pair of locations.

• **Facility-location planning**: eccentricity, network center, radius and diameter identify the best location for a facility such as a hospital or fire station.

Because all algorithms work on the same network, their results can be compared directly; for example, the automated tests check that the Floyd–Warshall and Dijkstra algorithms always report the same shortest distances.

04. FEATURES / SCOPES

The following features have been implemented and tested in the final version of the program. Each feature was added as a separate increment (Section 05). Possible extensions that are not implemented are listed separately under Future Works (Section 11.2).

04.1. Location Management

- Add a location by name. The system assigns a unique ID automatically (1, 2, 3, …).
- View all locations in a table showing their ID, name and number of roads
- Update (rename) and delete locations. Deleting a location also deletes every road connected to it.
- A location name must not be empty, must not contain a comma and must not be repeated. Spaces at the start and end are removed, and duplicate names are detected ignoring upper/lower case.
- Location IDs are never reused, even after a location is deleted and the program is restarted.

04.2. Road Management

- Add a two-way road between two existing locations, with its distance in km.
- View all roads in a table showing both locations and the distance.
- Update a road's distance and delete roads.
- Validation rules: both locations must exist, a road cannot connect a location to itself, the distance must be greater than 0 and at most 10,000 km, and only one road is allowed between the same two locations.

04.3. City Network Display

- Displays the network as an adjacency list: each location followed by the locations it is directly connected to, with the road distances.

04.4. Network Exploration (BFS and DFS)

- The user chooses a start location.
- **BFS** visits locations level by level using a **queue**.
- DFS follows one path as far as possible before backtracking, using recursion.
- The program displays the visiting order, the number of locations reached and the locations that cannot be reached. Neighbors are visited in increasing ID order, so the result is always the same.
- Time complexity of both algorithms with an adjacency list: **O (V + E)**, where V is the number of locations and E is the number of roads.

04.5. Shortest Route (Dijkstra's Algorithm)

- The user selects a start location and a destination.

- Dijkstra's algorithm, using a min-priority queue, finds the route with the smallest total distance. The algorithm gives correct results because all road distances are positive.
- The program displays the full route as a sequence of locations, each road with its distance, and the total distance.
- A clear message is shown when the destination cannot be reached.
- Time complexity with an adjacency list: **O ((V + E) log V).**

04.6. Minimum Road Network (Kruskal's Algorithm)

- Kruskal's algorithm **sorts the existing roads** by distance and selects the shortest roads that do not form a cycle. A **Disjoint Set Union (DSU)** is used to detect cycles.
- The algorithm only selects from roads that already exist in the graph; it does not create new roads.
- If the network is **connected**, the result is a **Minimum Spanning Tree (MST)**: the set of roads with the smallest total length that connects all locations.
- If the network is **disconnected**, the result is a **Minimum Spanning Forest (MSF)**: one minimum spanning tree for each connected part of the network.
- The program displays the selected roads and their total length.
- Time complexity: **O (E log E)**, mainly for sorting the roads.

04.7. All-Pairs Shortest Paths (Floyd–Warshall Algorithm)

- Computes the shortest distance between **every pair** of locations by dynamic programming: dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]) for every intermediate location k.
- Uses a **distance matrix** and a **next-hop matrix**. Location IDs, which can have gaps after deletions, are mapped to matrix rows.
- Displays the distance matrix for small networks ("--" means no route) and rebuilds the shortest route between any two chosen locations, with every road and the total distance.
- Unreachable pairs are kept as infinity, so no route or road is ever invented. The distances always agree with Dijkstra's algorithm.
- Time complexity: **O(V^3)**; memory: **O(V^2)**.

04.8. Best Facility Location (Network Center)

- Uses the Floyd–Warshall distances to compute each location's **eccentricity**: the distance to the location farthest away from it.
- The **network center**, the location(s) with the smallest eccentricity (the **radius**), is suggested as the best place for a facility such as a hospital, fire station or emergency center. Equally good locations are all shown.
- The **diameter**, the largest shortest distance in the network, is displayed with the pair(s) of locations that produce it.
- If the network is disconnected, the program reports that there is no single center and that the diameter is infinite, then analyzes each connected part separately. Isolated locations are listed separately and are never suggested as the best location.
- Time complexity: **O(V^2)** after Floyd–Warshall, because no shortest paths are calculated again.

04.9. CSV Data Storage

- Locations, roads and the next location ID are stored in three files: locations.csv (id,name), roads.csv (from,to,distance) and next_id.txt.

• The program loads the saved data when it starts and saves it automatically after every successful change, so the network is kept between program runs.
• Invalid lines in the files are skipped with a clear message that gives the file name and line number.
• On the first run, when none of the data files exists, a sample network of 10 Dhaka locations and 14 roads is loaded automatically. Existing data is never overwritten.

The sample distances are illustrative values, not actual measured distances.

04.10. Console Interface and Input Validation

• A menu-driven console interface with 15 options in three groups (Locations, Roads, Network & Algorithms) and an Exit option.
• A color dashboard theme on Windows terminals, with an automatic plain-text mode when colors are not available.
• All input is validated: menu choices, IDs and distances must be valid numbers; invalid input is rejected with a clear message and the program asks again.

04.11. Summary of Algorithms Used

The table below summarizes every algorithm used in the program, how it is used and its time complexity (V = number of locations, E = number of roads).

Algorithm	How it is used in SmartCity	Relevance to smart city planning	Time complexity
Breadth-First Search (BFS)	Visits locations level by level from a start location using a queue; shows the visiting order and the unreachable locations.	Checks which parts of the city can be reached from a location (connectivity).	O(V + E)
Depth-First Search (DFS)	Follows one path as deep as possible, then backtracks (recursion).	Explores reachability and the connected parts of the road network.	O(V + E)
Dijkstra's algorithm	Finds the route with the smallest total distance between two locations using a min-priority queue.	Shortest travel route between two places.	O((V + E) log V)
Kruskal's algorithm with Disjoint Set Union (DSU)	Sorts the roads by distance and keeps a road only if it joins two different groups (DSU with path compression and union by rank).	Smallest total road length that keeps all locations connected (minimum infrastructure); a forest for a disconnected network.	O(E log E)
Floyd–Warshall algorithm	Dynamic programming over a distance matrix and a next-hop matrix; all pairs at once.	Distance table between every pair of locations and the route between any two.	$O(V^3)$ time, $O(V^2)$ memory
Best facility location (eccentricity, center, radius, diameter)	Uses the Floyd–Warshall distance matrix; the center has the smallest eccentricity.	Best location for a hospital, fire station or emergency center; separate results for each connected part.	$O(V^2)$ after Floyd–Warshall

05. DEVELOPMENT MODEL

We have chosen **the Incremental Development Model** for this project.

In this model, the system is built in separate stages, where each stage adds a new working module to the existing system. Each module is designed, coded, and tested before the next one begins.

We chose this model for the following reasons:

- The project has clearly separate parts: graph and data management, network display, BFS and DFS, Dijkstra's algorithm, Kruskal's algorithm, the Floyd–Warshall algorithm and the facility-location analysis. Each part can be built and tested on its own.
- After every increment the program works and can be demonstrated, even if time becomes short near the end of the semester.
- Errors are easier to find and fix within a small part of the program.
- Team members can work on different parts at the same time.
- Each completed increment can be shown to the supervisor for feedback before the next one begins.

The project was developed in eight increments:

- **Increment 01:** Graph structure (adjacency list), location and road management (add/view), menu, network display and tests.
- **Increment 02:** Update/delete operations, CSV save/load and sample data.
- **Increment 03:** BFS and DFS network exploration.
- **Increment 04:** Dijkstra's shortest route.
- **Increment 05:** Kruskal's minimum spanning tree with Disjoint Set Union.
- **Increment 06:** Validation improvements (name and distance rules), code audit and documentation review.
- **Increment 07:** Color console interface and first-run sample data.
- **Increment 08:** Floyd–Warshall all-pairs shortest paths and best facility location.

Each increment was designed, coded and tested with the automated test suite, and committed to the dev branch of the GitHub repository before the next increment began.

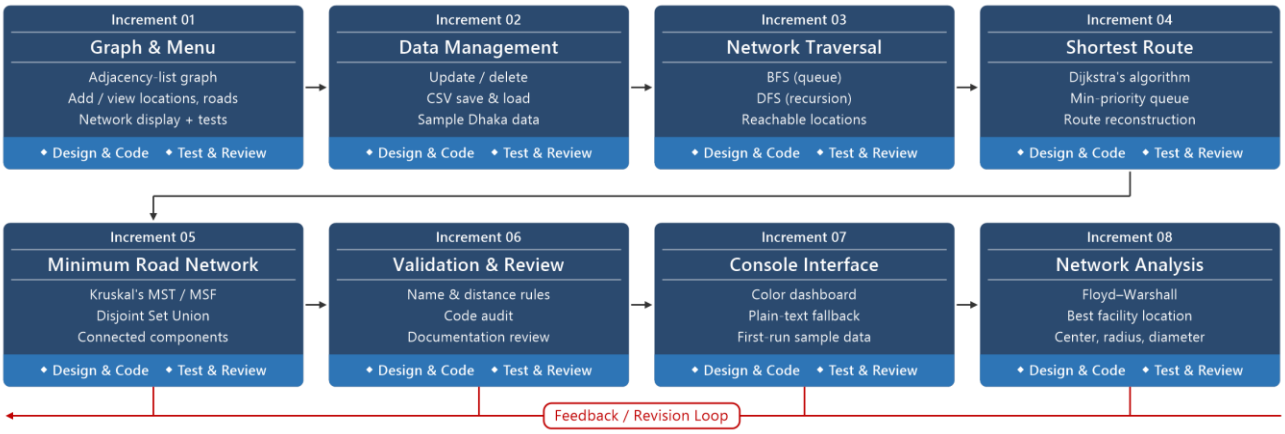


Figure 01: Incremental Development Model — Smart City Route & Infrastructure Planner

06. FEASIBILITY ANALYSIS & POSSIBLE RISKS

06.1. Feasibility Analysis

Technical Feasibility

The system uses C++17, the C++ Standard Template Library (STL) and simple file handling. The team members have studied programming and basic data structures in their coursework, and the algorithms used are standard topics in an algorithms course. The required tools (GCC from MinGW-w64, CMake and Visual Studio Code) are free and available on Windows. No paid software or special hardware is required.

Economic Feasibility

All software tools are free and open source. Development is done on the team members' existing personal computers. No server, hosting or software license is needed.

Operational Feasibility

The system is a menu-driven console application. Users choose options by number and type names and distances when asked, so only basic computer skills are needed.

06.2. Possible Risks

- **Incorrect Algorithm Results**: an error in an algorithm could produce wrong routes or totals. This will be reduced by automated tests that compare the results with answers worked out by hand on small example networks.
- **Data File Errors**: a missing or damaged CSV file could cause data loss or incorrect data. The program will check the files when loading and report any problems clearly.
- **Scope Creep**: the team may be tempted to add extra features. The core features were completed first; additional features (the Floyd–Warshall algorithm and the best facility location) were added only afterwards, as separate tested increments.
- **Team Coordination Issues**: four members working on different parts may make conflicting changes. This will be managed by agreeing on the shared graph structure first and by using Git and GitHub for version control.
- **Academic Schedule Conflicts**: exams and other coursework may reduce the time available. The incremental model ensures that a working program exists after each increment.

07. REQUIREMENTS

Hardware Requirements

- Any PC or laptop with at least 2 GB RAM.
- At least 1 GB free disk space for the compiler, tools, and project files.
- Standard keyboard and monitor.

Software Requirements

- Operating System: Windows 10 or Windows 11 (the development and testing platform).
- Compiler: GCC 16.2.0 from WinLibs MinGW-w64 (64-bit, POSIX threads, SEH, UCRT) with C++17 support. Other MinGW-w64 GCC versions with C++17 support can also be used.
- Build System: CMake 3.21 or newer, using MinGW Makefiles (Ninja is optional).
- Editor: Visual Studio Code with C/C++ extension.

- Version Control: Git and GitHub for source code management.

Programming Language

- C++ (C++17 standard).
- C++ Standard Library only, such as iostream, fstream, filesystem, string, vector, map, set, queue, algorithm, sstream, iomanip and cmath; the Windows console API is used for colors.

Data Storage

- Locations, roads and the next location ID are stored in text files (locations.csv, roads.csv and next_id.txt), read and written using C++ file streams (ifstream and ofstream).

No Internet Required

- The system is fully offline. No network access is needed at any point.

07.1. Source Code Repository

The project source code is maintained on GitHub for version control and team collaboration.

Repository Link: https://github.com/makhzumbinharun/smart-city-route-planner

GitHub will be used for:

- Source code management.
- Team collaboration.
- Progress tracking.
- Backup and version history.
- Documentation storage.

08. ACTIVITY AND TIME SCHEDULE/PROJECT BREAKDOWN WITH TIME ESTIMATION

The table below shows the estimated **total effort of the whole team** (not the effort of each member). All values are approximate.

#	Stages/ Tasks	Efforts, man-hours
Stage 1	**Analysis and Design**	
1.1	Requirements analysis and feature finalization	06
1.2	System design, module breakdown, and work plan	06
1.3	Graph data structure and CSV file format design	06
1.4	Flowchart and algorithm design	08
Stage 2	**Implementation**	
2.1	Graph structure, adding and viewing locations and roads, menu	16
2.2	Update/delete operations and CSV save/load	12
2.3	BFS and DFS traversal	10

2.4	Dijkstra's shortest route	12
2.5	Kruskal's minimum spanning tree with DSU	12
Stage 3	**Testing and other QA tasks**	
3.1	Unit testing for each module	10
3.2	Integration testing and bug fixing	08
Stage 4	**Deployment**	
4.1	Project report writing	04
4.2	Presentation and viva preparation	04
Total Estimated Efforts:		**114 man-hours**

09. BUDGET

This project does not require any financial investment. All tools, compilers, and software used are free and open source. The development is done on the team members' personal laptops and no external services are needed.

Project Item	Amount	Unit Price	Total
Software License (GCC, CMake, Visual Studio Code, Git)	N/A	N/A	N/A (Free)
Development Machine(s)	N/A	N/A	N/A (Personal)
Database or Server License	N/A	N/A	N/A
Training Fees	N/A	N/A	N/A
Internet / Communication	N/A	N/A	N/A
Office Supplies / Printing	300.00	N/A	300.00
Total			BDT 300.00

10. DEVELOPMENT TEAM & STAKEHOLDERS

10.1. Development Team

ID	Name	Designation	Responsibilities
20254103279	MAKHZUM-BIN-HARUN	Team Leader & Core Programmer	Team coordination, system design, graph data structure, menu program and integration
20254103245	MUNTAHA MOU JIM	Team Member	BFS and DFS implementation and testing
20254103248	OALID KHAN	Core Programmer & SQA	Dijkstra's and Kruskal's algorithm implementation and testing

20254103269	MD. RAFIUL ISLAM	Team Member	Update/delete operations, CSV file handling, testing and documentation, and Floyd–Warshall algorithm implementation and testing

10.2. Stakeholders

Stakeholder	Role in System	Interest
Students and learners	Main users	Want to see how graph algorithms explore a network, find shortest routes between two or all locations, select a minimum road network and suggest the best facility location on a simplified city model
Course Supervisor	Academic Evaluator	Reviews project progress and evaluate the correct use of algorithms and data structures
Development team	Developers	Want to gain practical experience in implementing and testing graph algorithms in C++

11. LIMITATIONS & FUTURE WORKS

11.1. Limitations

The following are known limitations of this version of the system:

- The program has a console-based interface only; there is no graphical user interface.
- The network is an abstract graph, not a geographical map. Location coordinates and real map data are not used.
- The sample road distances are illustrative values, not measured city data.
- Each road has only one weight: its distance in km. Travel time, fare, traffic and construction cost are not considered.
- All roads are undirected (two-way), and only one road is allowed between the same two locations.
- The application is single-user and does not support concurrent access.
- Windows is the target platform for development and testing. Other operating systems are not a project requirement.
- The best facility location chooses only among existing locations (not points along a road) and minimizes the distance to the farthest location; population, demand and cost are not considered.
- The full distance matrix is displayed only for small networks (about 10 locations); larger networks use the start/destination query.

11.2. Future Works

The following features are **not part of the current version**. They could be added in future versions if the system is extended:

- A graphical user interface with interactive map visualization (for example using Qt 6), reusing the same graph and algorithm code.

- Integration of the real-world Dhaka road network and real map data (location coordinates and measured road distances).
- Traffic-aware route optimization, using travel time and congestion instead of distance only.
- Emergency vehicle route planning, for example from the nearest facility to an incident location.
- Advanced facility-location models, such as placing several facilities (p-center and p-median problems) or weighting locations by population or demand.
- Real-time traffic and infrastructure data integration.
- Additional road attributes (fare, construction cost), one-way roads and multiple transport modes.
- Comparison with other algorithms, such as Prim's algorithm for minimum spanning trees, and infrastructure budget planning.

12. REFERENCES

[1]. T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA: MIT Press, 2022.

[2]. E. W. Dijkstra, "A note on two problems in connexion with graphs," Numerische Mathematik, vol. 1, pp. 269–271, 1959.

[3]. J. B. Kruskal, "On the shortest spanning subtree of a graph and the traveling salesman problem," Proceedings of the American Mathematical Society, vol. 7, no. 1, pp. 48–50, 1956.

[4]. R. W. Floyd, "Algorithm 97: Shortest path," Communications of the ACM, vol. 5, no. 6, p. 345, 1962.

[5]. S. Warshall, "A theorem on Boolean matrices," Journal of the ACM, vol. 9, no. 1, pp. 11–12, 1962.

[6]. R. E. Tarjan, "Efficiency of a good but not linear set union algorithm," Journal of the ACM, vol. 22, no. 2, pp. 215–225, 1975.

[7]. S. L. Hakimi, "Optimum locations of switching centers and the absolute centers and medians of a graph," Operations Research, vol. 12, no. 3, pp. 450–459, 1964.

[8]. B. Stroustrup, The C++ Programming Language, 4th ed. Boston, MA: Addison-Wesley, 2013.

[9]. R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner's Approach, 8th ed. New York, NY: McGraw-Hill Education, 2014. (Referenced for the Incremental Development Model.)

[10]. Free Software Foundation, GCC, the GNU Compiler Collection. Available: https://gcc.gnu.org

[11]. B. Sanders, WinLibs: standalone build of GCC and MinGW-w64 for Windows. Available: https://winlibs.com

[12]. Kitware, CMake Documentation. Available: https://cmake.org/documentation

[13]. Microsoft, Visual Studio Code Documentation. Available: https://code.visualstudio.com/docs